CETP — Low-Level Design

Cross-Environment Runtime Prediction for PostgreSQL Capstone project, PES University · Document revision 2026-10-13

> **This document supersedes the earlier LLD entirely.** Every headline figure in that version predates three corrections: a buffer-accounting bug in the dataset, a compute signal measured at the wrong degree of parallelism, and the per-operator rework. It also described a class inventory (`QueryProfiler` , `ConfidenceGate` , `MeasurementStore`) that did not exist. The corrections are itemised in `DOC_CORRECTIONS.md` . **Every number below is generated by a script into `results/` and cited to its file. Nothing here is typed by hand.**

---

1. Problem

A developer writes a query and runs it on their laptop. It takes 400 ms. Production holds the same schema on different hardware. **Will it meet the SLA there?**

Today the developer either guesses, or deploys and waits. The guess people actually make is *"production will behave like my machine"* — the **naive baseline**, and the one any predictor has to beat to be worth running.

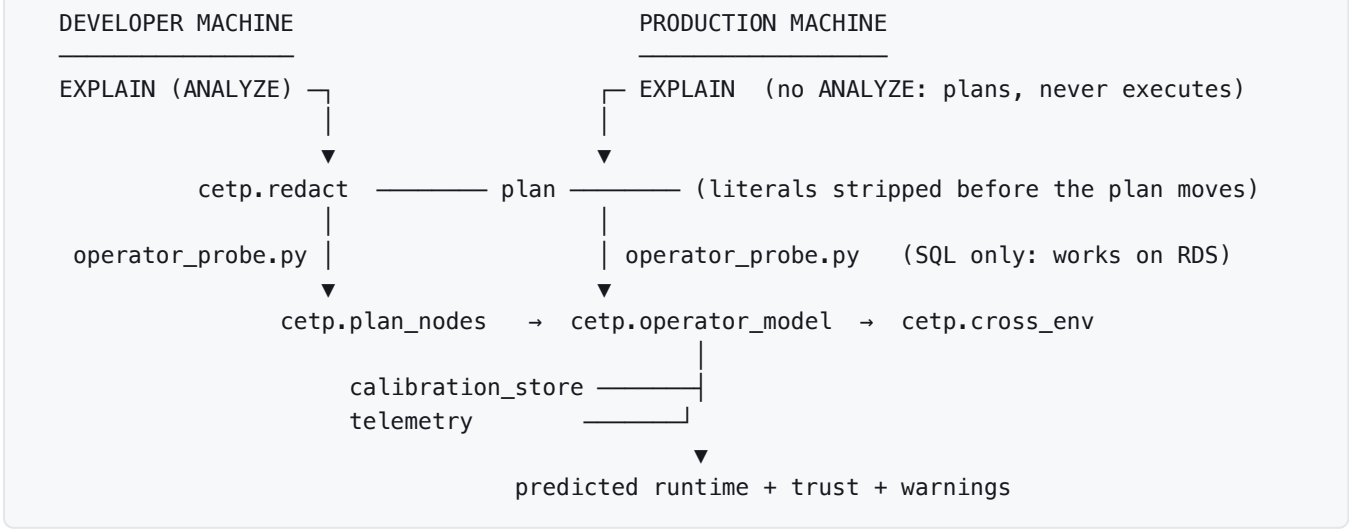
1.1 Scope

In scope. PostgreSQL. A single query. Both environments reachable for a one-off capability measurement. Latency percentiles as the decision variable.

Out of scope, and measured rather than assumed. Concurrency (supplied by the caller, §7.4). Differing data volumes (implemented, never validated, §9). Storage-class differences (no probe axis exists, §9). Query optimisation advice.

---

2. Architecture



2.1 Modules

Module	Responsibility
`cetp/plan_nodes.py`	Decompose an EXPLAIN plan into per-node self time; map node types to operator axes
`cetp/operator_model.py`	Weighted per-operator scaling, parallelism, cardinality, concurrency, calibration fitting
`cetp/cross_env.py`	Orchestration, signature loading, trust, warnings, cross-architecture refusal
`cetp/calibration_store.py`	Per-(dev, prod) correction state; the mechanism behind improvement with use
`cetp/telemetry.py`	Predicted-vs-actual logging; closes the loop into the calibration store
`cetp/ingest.py`	`auto_explain` and `pg_stat_statements` ingestion
`cetp/redact.py`	Strip literals before a plan leaves the machine; verify the strip
`cetp/predict.py`, `regress.py`, `signature.py`	Legacy two-scalar predictor, regression mode, signature registry
`cetp/cli.py`	`predict · regress · redact · signature · predict-prod · selftest`
`scripts/collection/operator_probe.py`	The capability probe
`scripts/collection/predict_prod.py`	End-to-end flow over two live connections

Runtime dependencies: none. The package imports only `json`, `math`, `re` and the standard library. The predictor ships a formula plus a calibration table rather than a pickled model, so it cannot go stale against a library version.

---

3. The capability probe

3.1 Why the previous signature was replaced

The original signature was two scalars from a C program: STREAM-triad memory bandwidth and FP-vector GFLOPS. Three measured problems:

Problem	Evidence
It did not transfer across instruction sets	arm64 laptop → x86 EC2 scored **324%** MAPE (**160.68%** after fixing the thread count) against **129.39%** for assuming production equals development — *worse than doing nothing* (`results/laptop_to_ec2.md`)
Its two axes were not two axes	r = **+0.958**, 97.9% of variance on one principal component, tracking core count at r = +0.841 (`results/tier1_hw_signature.md`)
It measured the wrong quantity	TPC-H is hash probes, sorts, aggregation and scans. Nothing in the workload issues vector FP instructions.
It could not describe managed PostgreSQL	It needs a shell and a compiler. RDS, Aurora and Cloud SQL give neither.

3.2 What replaced it

Six axes, each measured by running the operator in **SQL**:

Axis	Measures
`seqscan`	tuples/s through the scan and filter path
`hashjoin`	tuples/s to build and probe a hash table
`sort`	tuples/s through the sort path
`hashagg`	tuples/s through hash aggregation
`nestloop`	tuples/s through indexed nested-loop lookup (random access, TLB)
`parallel_seqscan`	the same scan with parallelism on, yielding `parallel_speedup`

Being only SQL, it runs anywhere PostgreSQL does, including managed services.

3.3 Two guards, each of which caught a real fault

Operator verification. Every probe asserts the planner actually chose the operator it names. This caught the `nestloop` probe silently measuring a hash semi-join and reporting 5.2 M tuples/s instead of the true 1.25 M.

Repeatability gate. A capability probe is only a capability measurement on an idle machine. The gate uses the **bootstrap standard error of the median**, threshold 0.08.

> The first version gated on the coefficient of variation, which was the wrong statistic: the reported rate is a *median*. Parallel axes jitter run-to-run (cv **0.587**) while their medians reproduce to **3%** (between-pass cv 0.027). Gating on cv failed 4 of 8 good machines, shifted the usable population between runs, and moved headline numbers by 5–15 points for reasons unrelated to the hardware.

3.4 Probe sizing

Probe tables are 2 M and 400 k rows, with 9 repetitions (11 for the parallel axis) and the first discarded as warm-up.

> Sizing is load-bearing. At 400 k rows the parallel scan finished in **18 ms**, so worker startup dominated: cv 0.50 and an **impossible 1.81× speedup on a single-CPU quota**. At 2 M rows the same machine reports **0.808×** — parallelism correctly making it *slower* when three workers share one core.

---

4. Plan decomposition

4.1 Two corrections to `Actual Total Time`

PostgreSQL's `Actual Total Time` is two things at once, and both must be undone:

1. **Cumulative** — it includes all descendants. Self time is inclusive minus children.
2. **Per loop** — an inner node reports the *average* of one loop. Total is `Actual Total Time × Actual Loops`. An index scan reporting 0.31 ms over 485 loops cost 150 ms.

4.2 Parallelism keys on `under_gather`, not `Parallel Aware`

`Parallel Aware` marks only nodes that *partition* work. Everything **below a** `Gather` runs once per participant. On TPC-H q19 the `Nested Loop`, inner `Aggregate` and `Index Scan` all report `loops=3` while only the `Seq Scan` is `Parallel Aware`.

Keying the parallelism correction on	predicted scaling	error vs the true 3.011×
nothing (serial axes only)	1.080	64.1%
`Parallel Aware`	1.493	50.4%
`under_gather`	**2.664**	**11.5%**

4.3 Node-type mapping

`Seq Scan` → `seqscan`; `Hash Join`, `Hash` → `hashjoin`; `Sort`, `Incremental Sort`, `Merge Join` → `sort`; `Nested Loop`, `Index Scan`, `Index Only Scan`, `Bitmap Heap Scan`, `Bitmap Index Scan`, `BitmapAnd`, `BitmapOr` → `nestloop`; `Aggregate` splits by `Strategy` (Hashed→ `hashagg`, Sorted→ `sort`, Plain→ `seqscan`). Control-flow nodes map to nothing and their time is reported as an **unmapped residual** rather than hidden.

---

5. The model

```
scaling = Σ over (axis, parallel?)  w · [rate_dev(axis)·s_dev] / [rate_prod(axis)·s_prod] · card
T_prod  = T_dev × scaling × calibration × N^e
```

`w` is development's **wall-time** share for that bucket and sums to 1.

5.1 Why a weighted ratio, not a sum of scaled node times

Under a `Gather`, node self-times sum to total **CPU** time — measured at **2.85× wall** on a parallel TPC-H q19. Summing scaled self-times would predict roughly 3× too high. Normalising to weights makes that factor cancel. The price is that the model predicts a *ratio* and needs a measured `T_dev`, which the product has anyway.

5.2 Cardinality

Linear in the row ratio for scans, joins and hash aggregation; for sorts the exact n-log-n ratio `r·(1 + log r / log n)`.

> The first implementation used `r·log₂(2r)`, giving **12.0×** at r = 4 against a true **≈4.5×** — a 2.7× over-penalty on every sort. **This term is implemented but never validated**: every environment measured holds identical data, so it is only ever exercised at a ratio of 1.0.

---

6. Results

All figures from `results/`, regenerated by `make results`, `make operator-results` and the evaluation scripts.

6.1 Capability grid — 1 232 pairs, 8 environments

Predictor	MAPE
naive (prod = dev)	72.52%
one probe axis	73.60%
geometric mean of serial axes	76.29%
per-operator, **no** parallelism axis	76.21%
one scalar + parallelism applied whole-query	29.90%
one scalar + parallelism **localised**	24.58%
full per-operator model	**24.95%**

Paired cluster bootstrap, resampling whole production machines:

Comparison	Δ MAPE	95% CI	
naive – operator	+47.57 pp	[+38.80, +56.93]	significant
no-parallelism – operator	+51.27 pp	[+40.07, +63.93]	significant
whole-query – localised parallelism	+5.32 pp	[+0.14, +12.04]	significant
localised scalar – full operator model	**−0.37 pp**	**[−0.84, +0.01]**	**not significant**

Reading this honestly: the parallelism term and its localisation to the plan's parallel part are the result. Per-operator *rate* weighting adds nothing measurable. One serial capability scalar, correctly localised, matches the full model. This is a *simplification* finding and is reported as one.

6.2 Signature dimensionality

	old 2-scalar	operator rates
PC1 share of variance	97.9%	**59.6%**
mean abs. correlation between axes	0.958	**0.420**
effective rank at 95% variance	1	**3**

6.3 Laptop → production

Population	naive	operator model
same architecture, 7 machines, 154 pairs	37.39%	**29.31%**
cross architecture, 4 x86 machines, 88 pairs	**21.74%**	**68.92%**
all 11 machines, 242 pairs	31.70%	43.72%

Within an architecture the model works. The gain concentrates where capability genuinely differs: on the most constrained machine, 62.58% → 23.47%.

Across architectures it fails, reproducibly — on all four x86 machines (66.15 / 67.26 / 69.44 / 72.84%), with the four signatures agreeing within 5% of each other, so this is systematic rather than noise.

6.4 Why cross-architecture fails

Not operator mix. Measured directly: arm64 → x86_64 perturbs the operator mix by **1.365×**, while the worst pair *within* one arm64 host perturbs it by **1.371×**.

What fails is the **absolute scale**: the probe reported the x86 machine **1.667× slower** than the laptop when it was actually **1.044×**. Tuples/s on a synthetic operator is not comparable across microarchitectures — the same defect as the FP-GFLOPS signature it replaced.

6.5 Calibration

Queries measured on production	cross-ISA MAPE
0 (zero-shot)	72.84%
3	23.86%
5	23.38%
floor for any single-factor model on that pair	*16.75%*

Calibrated, the cross-architecture model reaches **parity** with assuming production equals development (23.38% against 21.41%) — it does not beat it. **The system therefore refuses an uncalibrated cross-architecture prediction** rather than returning a number measured to be 3.2× worse than doing nothing.

6.6 A ceiling on this entire model class

Every predictor in this lineage multiplies dev runtime by one number per machine pair. On the EC2 data the scaling factor varies **18–43× across queries on one machine**, which puts a floor of **59.7%** MAPE on any such model. The best measurement reached 55.88% — already at the floor.

---

7. Supporting mechanisms

7.1 Redaction

`EXPLAIN` output embeds literal values, which on a real database are user data. Two levels: `structural` replaces literals with `?`; `full` additionally hashes identifiers. `assert_clean()` re-scans the output and raises if a literal survives.

> **Security fix, 2026-10-13.** `auto_explain` emits a `Query Text` field containing the entire statement with every literal. The redactor passed it through untouched — nothing had exercised that path, because hand-captured EXPLAIN output omits the field. It is now redacted like an expression and dropped entirely at `full`.

7.2 Calibration state

One correction per (dev, prod) pair, fitted as the **median** of actual/predicted, stored JSON, written atomically, marked *provisional* below 3 observations.

> Median, not mean, and provisional below 3, because **one arbitrary observation measured worse than no calibration at all** (19.51% against 15.84% uncalibrated).

What improves with use, precisely: accuracy for an environment *pair*, as observations of that pair accumulate. **Not** with the number of machines in the registry — a correction absorbs one machine's scale error and says nothing about another.

7.3 Telemetry

One JSON line per prediction, resolved later against the measured duration, reporting MAPE **and the naive baseline alongside it**. Refuses to log query text, SQL or plans.

7.4 Concurrency

`T(N) = T(1)·N^e`. Median elasticity **0.734**, range 0.286–0.984.

	N=2	N=4	N=8
ignore concurrency	33.90%	56.09%	73.79%
global default e = 0.734	24.34%	35.73%	41.81%
oracle (query's own e)	26.29%	25.38%	6.72%

Elasticity is not predictable from a plan — leave-one-query-out, plan features score MAE 0.191 against 0.167 for simply using the median. The load factor is therefore a caller- supplied input, and `prod_clients=1` emits a loud warning.

---

8. Validation

Test suite	**108 tests**
`cetp selftest`	redaction, prediction, regression, calibration, operator path
Determinism	`make verify-determinism` runs the canonical evaluation twice, asserts byte-identical output
PostgreSQL versions	**14, 15, 17, 18** — 88 plans, 100% decompose and redact cleanly, max unmapped 8.4% (`results/version_compat.json`)
CI	gate on example plans; x86 collection on GitHub Actions

PostgreSQL 13 and 16 are not tested and are not claimed.

Node fields that differ across versions: `Disabled` , `Full-sort Groups` , `Index Searches` , `Merge Cond` , `Presorted Key` (18 only); `Params Evaluated` (14/15 only). All parse.

---

9. Known limitations

1. **Cross-architecture prediction requires calibration** and then only reaches parity with the naive baseline.
2. **Per-operator rate weighting is not justified** by the evidence (−0.37 pp, CI crossing zero). The parallelism term carries the result.
3. **The cardinality term is unvalidated** — only ever exercised at a data ratio of 1.0.
4. **No storage axis exists.** It is the diagnosed cause of the residual bias (over-prediction correlates with per-query I/O fraction, Spearman r = +0.302). An axis was built and does *not* measure storage: it tracks CPU quota (+0.762) over memory (+0.548), so the model does not use it.
5. **Concurrency** is an input, not a prediction.
6. **22 TPC-H queries, SF1, single client, one engine version per environment.**
7. **Environments are 8 containers on one arm64 host plus 4 AMD CI runners** — not a production fleet.

8. **The legacy two-scalar predictor is still what `cetp predict` and the VS Code extension use.** The per-operator path is `cetp predict-prod`.

---

10. Reproducing every figure

```
pip install -e .            # no runtime dependencies
cetp selftest
make results                # results/CANONICAL.md
make probe-here             # capability signature for this machine (run it IDLE)
make grid-setup             # load TPC-H SF1 into a Docker volume, once
make grid-collect           # sweep the 4x2 capability grid
make operator-results       # results/operator_model.md
python3 scripts/analysis/laptop_to_prod_evaluation.py
python3 scripts/analysis/laptop_to_ec2_evaluation.py
python3 scripts/analysis/concurrency_model.py
python3 scripts/analysis/version_compat_check.py
python3 scripts/docs/build_pdf.py docs/LLD.md
```

Section	Source
§3, §6.4	`results/cross_isa_operator_rates.md`
§6.1, §6.2	`results/operator_model.md`, `operator_model_metrics.json`
§6.3, §6.5	`results/laptop_to_prod.md`, `cross_isa_prediction.md`
§6.6	`results/laptop_to_ec2.md`
§7.4	`results/concurrency.md`
§8	`results/version_compat.json`
Legacy predictor	`results/CANONICAL.md`